ds that use attributes

Today

Welcome to your next lesson: Self param, and methods that use attributes.

I'm preparing your personalized lesson based on what you've learned, what's next, and the best way to help you grow.

Step by step, you’re moving forward.

Imagine you have a Pet class with a method that should print the pet’s name and species.

You don’t want to write a separate method for every pet (luna_describe, max_describe, etc.). Instead, you write one method, and it should automatically use the right pet’s data when you call it.

That’s where self comes in: inside the method, self is “the pet that called this method,” so self.name means “the name of that specific pet.” 🐶🐱

To start us off: conceptually, which of these do you feel is closer to the truth?

self is:
A) The class (Pet) itself
B) The specific object (like luna or max) that called the method

You can just answer with A or B.

b

Nice, B is exactly right 🎯

Inside a method, self is the specific object that called it — like luna or max.

Let’s make that more concrete.

Imagine this code:

Python
```
class Pet:
    def describe(self):
        print(self.name)
```
And later:

Python
```
luna.describe()
max.describe()
```
Step by step idea:

When luna.describe() runs → inside describe, self is luna
When max.describe() runs → inside describe, self is max

So self.name means:

luna.name in the first call
max.name in the second call
Does that mapping from luna.describe() → self is luna make sense so far, or should I walk through it with a tiny visual next?

That makes sense

Great. Let’s lock in how self connects the caller to the method with a simple visual 👇

Think of this flow when you call a method:

Text
```
luna.describe()
       |
       v
describe(self)
       |
```

```
       v
self is luna
self.name is luna.name
Same method, different caller:

Text
max.describe()
      |
      v
describe(self)
      |
      v
self is max
self.name is max.name
So the only thing that changes is: which object becomes self.

In your own words (short is fine):

When max.describe() runs, what does self.name refer to?

max.name

Exactly: self.name → max.name when you call max.describe() ✅

Now let’s add an attribute and see self actually use it.


Step by step example:

Python
class Pet:
    def describe(self):
        print(self.name + " is a " + self.species)

# imagine we already did something like:
# luna.name = "Luna"
# luna.species = "cat"
# max.name = "Max"
# max.species = "dog"
Key idea (one thing at a time):


Inside describe,
self.name = “the name of whoever called this”
self.species = “the species of whoever called this”

So:
```

luna.describe() will read luna.name and luna.species
max.describe() will read max.name and max.species
Quick check (no code, just thinking):

If max.name = "Max" and max.species = "dog", what text do you expect max.describe() to print?

Max is a dog

Exactly: Max is a dog ✅

That’s self in action: when max.describe() runs, self is max, so:

self.name → "Max"
self.species → "dog"
Now let’s see how changing an attribute affects the method.

Imagine this sequence:

Python
max.name = "Max"
max.species = "dog"

max.describe()   # uses current values

max.species = "wolfdog"

max.describe()   # uses updated values
The method describe didn’t change at all — only max.species changed.

So the second time we call max.describe(), it reads the new value.

Question: after we change max.species = "wolfdog", what do you expect the second call to max.describe() to print?

Max is a wolfdog

What do you think? Take your time and pick the best answer 👇

Luna and Max both use the same describe() method. If self.name appears in the method, whose name does it read when max.describe() runs?

A.
Luna’s name

B.
Max’s name

C.
The class’s name

D.
No name

Nice job, Ron 🎉 You nailed it: when max.describe() runs, self.name reads Max’s name.

Reason in one line:

The object that calls the method becomes self.
Here the caller is max, so self is max, and self.name is max.name.

So same method, different caller → self switches to that specific object.

You’re doing really well with self and attributes 💪

Do you want to keep practicing with a small coding task in the editor, or are you ready to wrap up this lesson and move on to the next one?

Let's try a coding task

Love it 🔥 Let’s do something small and focused.

We’ll practice this idea: one method, different objects, each

using its own attributes via self.

Here’s what I want you to do in the code editor (not in chat):

Define a Pet class with a describe method that:

Uses self.name and self.species
Prints something like:
Luna is a cat or Max is a dog
Create two pets, for example:

One cat named "Luna"
One dog named "Max"
Call describe on both objects:

```
luna.describe()
max.describe()
```

Don’t worry about how the attributes get onto the objects; use any way you’ve learned that works in this course so far.

Go ahead and:

Open the editor (</> button)
Write the code for the class, the two pets, and the two method calls
Hit Run to check it
Then hit Submit code

I’ll wait here and review it when you submit 🚀

Python

```
class Pet:
    def describe(self):
        print(f"{self.name} is a {self.species}")

luna = Pet()
max = Pet()
luna.name = "Luna"
luna.species = "cat"
luna.describe()
max.name = "Max"
max.species = "dog"
max.describe()
```

Nice work, Ron — this is clean, idiomatic Python ✅

You did all the key things right:

Defined describe with self and used self.name + self.species
Created two separate objects: luna and max
Gave each its own attributes
Called the same method on both, and got:
Luna is a cat
Max is a dog

That shows you fully get: the same method, different objects, each using its own data via self 🎯

Let’s go one step deeper with attribute changes.

In the same file, right after your current code, add this:

Change luna.species to something else, like "tiger".
Call luna.describe() again.

Example idea (don’t paste in chat, do it in the editor):

change the attribute
call describe again

Then:

First, before running, think: “What will the new luna.describe() print?”
Run your code.
Then hit Submit code again so I can see the new output.

Go for it 🧠💻

Python
```
class Pet:
    def describe(self):
        print(f"{self.name} is a {self.species}")

luna = Pet()
max = Pet()
```

```
luna.name = "Luna"
luna.species = "cat"
luna.describe()
max.name = "Max"
max.species = "dog"
max.describe()
luna.species = "tiger"
luna.describe()
```

Nice, Ron — that code is spot on again ✅

You just proved an important idea:

First luna.describe() → Luna is a cat
Then you changed luna.species to "tiger"
Next luna.describe() → Luna is a tiger

The method code didn't change at all. It simply read self.species again, and since self is luna, it picked up Luna's new species value.

So: methods always use the object's current attributes at the moment they run 🧠

One last quick check about independence of objects:

Right after your existing code, add this in the editor:

Python

```
print("Now describing Max again:")
max.describe()
```

Then:

Before you run it, think: will Max's line change or stay the same?
Run and Submit code.
Tell me (in words): did Max's description change, or stay as Max is a dog?
It stays the same

Perfect ✅

That’s exactly what should happen:

You changed luna.species to "tiger" → only Luna changed.
max kept its own attributes, so max.describe() still prints Max is a dog.

So you’ve got all three key ideas:

self is the object that called the method (luna, max, etc.).
Methods use attributes through self.attribute (like self.name, self.species).
Methods always use the object’s current attribute values, and each object stays independent.

You’re solid on this 💪

Before we wrap this lesson:

Anything about self and attributes that still feels even a bit fuzzy, or are you good to move on to the next lesson?

I’m good to move on

Nice work, Ron 🎉 You’ve fully mastered this lesson.

Quick recap in plain language:

What self really is: inside an instance method, self is the specific object that called the method (luna, max, etc.).
How methods use attributes: you correctly wrote methods that use self.name, self.species, etc., so the same method works for different pets.
Current values + independence: you showed that:
Changing luna.species changed only Luna’s output.
Max’s output stayed the same, because each object keeps its own data.

You’re ready for the next step in OOP 🚀

When you’re set, hit Next Lesson in the Learn tab, and we’ll keep building from here.

```

class Pet:
    def describe(self):
        print(f"{self.name} is a {self.species}")

luna = Pet()
max = Pet()
luna.name = "Luna"
luna.species = "cat"
luna.describe()
max.name = "Max"
max.species = "dog"
max.describe()
luna.species = "tiger"
luna.describe()
```